LAPORAN TUGAS PEKAN 6

STRUKTUR DATA

Disusun oleh:

FAHRI SYAH PUTRA RAMADHAN

2511531015

Dosen pengampu:

Dr. Wahyudi. MT

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

PADANG, 2026

```java
package pekan6_2511531015;

public class NodeDLL_2511531015 {
    //mendefinisikan  kelas NOde
    int data_1015; //data
    NodeDLL_2511531015 next_1015; // Pointer ke next mode
    NodeDLL_2511531015 prev_1015; // pointer ke previous mode

    //konstruktor
    public NodeDLL_2511531015(int data) {
        this.data_1015 = data;
        this.next_1015 = null;
        this.prev_1015 = null;
        }
    }

```

NodeDLL_2511531015.

The code in the screenshot defines a Java class NodeDLL_2511531015, which represents a node in a **doubly linked list** (DLL). Here's a breakdown of the class structure and constructor:

1. **Attributes**:
    - int data_1015: Holds the data for the node.
    - NodeDLL_2511531015 next_1015: A pointer (reference) to the next node in the list, initialized to null.
    - NodeDLL_2511531015 prev_1015: A pointer (reference) to the previous node in the list, also initialized to null.
2. **Constructor** (NodeDLL_2511531015(int data)):
    - The constructor initializes the data_1015 attribute with the passed data parameter.
    - next_1015 is set to null, as this node is not connected to any node initially.
    - prev_1015 is also set to null, as this node doesn't have a previous node yet.

This class is a building block for a doubly linked list, where each node points to both its next and previous nodes, allowing for efficient traversal in both directions.

```java
package pekan6_2511531015;

public class InsertDLL_2511531015 {
    //menambahkan node di awal DLL
    static NodeDLL_2511531015 insertBegin(NodeDLL_2511531015 head, int data) {
        //buat node baru
        NodeDLL_2511531015 new_node = new NodeDLL_2511531015(data);
        //jadikan pointer nextnya head
        new_node.next_1015 = head;
        //jadikan pointer prev head ke new_node
        if (head != null) {
            head.prev_1015 = new_node;
        }
        return new_node;
                }
    //fungsi menambahkann node di akhir
    public static NodeDLL_2511531015 insertEnd(NodeDLL_2511531015 head, int newData) {
        //buat node baru
        NodeDLL_2511531015 newNode =  new NodeDLL_2511531015(newData);
        //jika dll null jadikan head
        if (head == null) {
            head = newNode;
        }
        else {
            NodeDLL_2511531015 curr = head;
            while(curr.next_1015 != null) {
                curr = curr.next_1015;

            }
            curr.next_1015 =newNode;
            newNode.prev_1015 = curr;

        }
        return head;
    }
    //fungsi menambahkan node di posisi tertentu
    public static NodeDLL_2511531015 insertAtPosition(NodeDLL_2511531015 head, int pos, int new_data) {
        //buat node baru
        NodeDLL_2511531015 new_node = new NodeDLL_2511531015(new_data);
        if (pos == 1) {
            new_node.next_1015 = head;
            if (head != null) {
                head.prev_1015 = new_node; }
            head = new_node;
            return head; }
            NodeDLL_2511531015 curr = head;
            for (int i = 1; i < pos - 1 && curr != null; ++i) {
                curr = curr.next_1015; }
```

```java
            }

            public static void printList(NodeDLL_2511531015 head) {
                NodeDLL_2511531015 curr = head;
                while (curr != null) {
                    System.out.print(curr.data_1015 + " <-> ");
                    curr = curr.next_1015;
                }
                System.out.println();

    }
            public static void main(String[] args) {
                //membuat dll 2 <-> 3 <-> 5
                NodeDLL_2511531015 head = new NodeDLL_2511531015(2);
                head.next_1015 = new NodeDLL_2511531015(3);
                head.next_1015.prev_1015 = head;
                head.next_1015.next_1015 = new NodeDLL_2511531015(5);
                head.next_1015.next_1015.prev_1015 = head.next_1015;
                //cetak DLL awal
                System.out.print("DLL awal: ");
                printList(head);
                //tambah 6 di akhir
                System.out.print("simpul 1 ditambah di awal: ");
                printList(head);
                int data = 6;
                head= insertEnd(head, data);
                printList(head);
                //menambah node 4 di posisi 4
                System.out.print("tambah node 4 di posisi 4: ");
                int data2 = 4;
                int pos = 4;
                head = insertAtPosition(head, pos, data2);
                printList(head);
            }
}
```

InsertDLL_2511531015.

1. Package Declaration

package pekan6_2511531015;

Baris ini mendeklarasikan package tempat kelas ini berada, yaitu pekan6_2511531015.

2. Class InsertDLL_2511531015

Kelas InsertDLL_2511531015 berfungsi untuk melakukan operasi pada **Doubly Linked List (DLL)**, yaitu menambah node di awal, di akhir, dan pada posisi tertentu dalam DLL.

2.1 Insert Node at the Beginning (insertBegin)

Fungsi ini digunakan untuk menambah node baru di awal DLL.

```
static NodeDLL_2511531015 insertBegin(NodeDLL_2511531015 head, int data) {
    NodeDLL_2511531015 new_node = new NodeDLL_2511531015(data);
    new_node.next_1015 = head;  // Menunjuk ke head lama
    if (head != null) {
        head.prev_1015 = new_node;  // Menunjuk balik ke node baru
    }
    return new_node;  // Node baru menjadi head
}
```

- **Langkah-langkah:**
    1. Membuat node baru (new_node) dengan data yang diberikan.
    2. Menetapkan pointer next_1015 pada node baru ke node head lama.
    3. Jika head tidak null, kita set pointer prev_1015 dari node lama untuk menunjuk ke node baru.
    4. Mengembalikan node baru yang menjadi head dari DLL.

2.2 Insert Node at the End (insertEnd)

Fungsi ini menambahkan node baru di akhir DLL.

```
public static NodeDLL_2511531015 insertEnd(NodeDLL_2511531015 head, int newData) {
    NodeDLL_2511531015 newNode =  new NodeDLL_2511531015(newData);
    if (head == null) {
        head = newNode;  // Jika DLL kosong, node baru menjadi head
    } else {
        NodeDLL_2511531015 curr = head;
        while(curr.next_1015 != null) {
            curr = curr.next_1015;  // Menemukan node terakhir
```

```
        }
        curr.next_1015 = newNode;  // Menghubungkan node terakhir dengan node baru
        newNode.prev_1015 = curr;  // Menghubungkan node baru dengan node terakhir
    }
    return head;  // Mengembalikan head yang tetap
}
```

- **Langkah-langkah:**
    1. Membuat node baru (newNode).
    2. Jika DLL kosong (head == null), node baru menjadi head.
    3. Jika DLL tidak kosong, lakukan traversal hingga node terakhir.
    4. Menambahkan node baru di akhir dengan mengatur pointer next_1015 dan prev_1015.
    5. Mengembalikan head yang tetap.

2.3 Insert Node at a Specific Position (insertAtPosition)

Fungsi ini menambahkan node baru pada posisi tertentu dalam DLL.

```
public static NodeDLL_2511531015 insertAtPosition(NodeDLL_2511531015 head, int pos, int new_data) {
    NodeDLL_2511531015 new_node = new NodeDLL_2511531015(new_data);
    if (pos == 1) {  // Jika posisi 1, insert di awal
        new_node.next_1015 = head;
        if (head != null) {
            head.prev_1015 = new_node;
        }
        head = new_node;
        return head;
    }
    NodeDLL_2511531015 curr = head;
    for (int i = 1; i < pos - 1 && curr != null; ++i) {
```

```
        curr = curr.next_1015;  // Menemukan posisi sebelumnya
    }
    if (curr == null) {
        System.out.println("posisi tidak ada");
        return head;
    }
    new_node.prev_1015 = curr;
    new_node.next_1015 = curr.next_1015;
    curr.next_1015 = new_node;
    if (new_node.next_1015 != null) {
        new_node.next_1015.prev_1015 = new_node;  // Menyambung node setelahnya
    }
    return head;  // Mengembalikan head yang tetap
}
```

- **Langkah-langkah:**
    1. Membuat node baru (new_node).
    2. Jika posisi yang diminta adalah 1, node baru langsung menjadi head.
    3. Jika posisi lebih dari 1, kita lakukan traversal untuk mencapai posisi yang diinginkan.
    4. Menyisipkan node baru di posisi tersebut dengan mengatur pointer next_1015 dan prev_1015.
    5. Jika posisi tidak valid, fungsi akan menampilkan pesan error dan mengembalikan head yang tidak berubah.

2.4 Print List (printList)

Fungsi ini digunakan untuk mencetak seluruh isi DLL.

```
public static void printList(NodeDLL_2511531015 head) {
    NodeDLL_2511531015 curr = head;
    while (curr != null) {
```

```
        System.out.print(curr.data_1015 + " <-> ");
        curr = curr.next_1015;  // Melakukan traversal hingga akhir list
    }
    System.out.println();
}
```

- **Langkah-langkah:**
    1. Melakukan traversal dari head hingga akhir DLL.
    2. Mencetak nilai data_1015 setiap node yang dilewati.
    3. Setelah mencapai akhir, mencetak baris baru.

3. Fungsi main

```
public static void main(String[] args) {
    NodeDLL_2511531015 head = new NodeDLL_2511531015(2);
    head.next_1015 = new NodeDLL_2511531015(3);
    head.next_1015.prev_1015 = head;
    head.next_1015.next_1015 = new NodeDLL_2511531015(5);
    head.next_1015.next_1015.prev_1015 = head.next_1015;
    System.out.print("DLL awal: ");
    printList(head);

    int data = 6;
    head = insertEnd(head, data);  // Menambah node 6 di akhir
    System.out.print("Simpul 1 ditambah di akhir: ");
    printList(head);

    int data2 = 4;
    int pos = 4;
    head = insertAtPosition(head, pos, data2);  // Menambah node 4 pada posisi 4
```

```
    System.out.print("Tambah node 4 di posisi 4: ");
    printList(head);
}
```

- **Langkah-langkah:**
    1. Membuat DLL dengan node awal: 2 <-> 3 <-> 5.
    2. Mencetak DLL awal.
    3. Menambah node 6 di akhir DLL.
    4. Menambah node 4 pada posisi ke-4.
    5. Mencetak DLL setelah setiap operasi.

4. Hasil Output

- **Output pertama** adalah list awal: 2 <-> 3 <-> 5.
- **Output kedua** setelah menambah node 6 di akhir: 2 <-> 3 <-> 5 <-> 6.
- **Output ketiga** setelah menambah node 4 di posisi ke-4: 2 <-> 3 <-> 5 <-> 4 <-> 6.

5. Kesimpulan

Kode ini mengimplementasikan operasi dasar pada **Doubly Linked List** (DLL) di Java, yang memungkinkan kita untuk:

- Menambahkan node di awal (insert at the beginning).
- Menambahkan node di akhir (insert at the end).
- Menambahkan node pada posisi tertentu (insert at a specific position).
- Mencetak isi DLL.

DLL memberikan keuntungan dalam hal traversal dua arah dan kemampuan untuk menghapus node dengan efisien, terutama jika kita memiliki akses ke node sebelumnya.

```java
package pekan6_2511531015;

public class PenulusuranDLL_2511531015 {
//fungsi penelusuran maju
    static void forwardTraversal (NodeDLL_2511531015 head) {
        //memulai penelusuran dari head
        NodeDLL_2511531015 curr = head;
        //lanjutkan sampai akhir
        while (curr != null) {
            //print data
            System.out.print(curr.data_1015 + " <-> ");
            //pindah ke mode berikutnya
            curr = curr.next_1015 ;

        }
        //print spasi
        System.out.println();
    }
    //fungsi penelusuran mundur
    static void backwardTraversal(NodeDLL_2511531015 tail) {
        //mulai dari akhir
        NodeDLL_2511531015 curr = tail;
        //lanjut sampai head
        while (curr != null) {
            //cetak data
            System.out.print(curr.data_1015 + " <-> ");
            //pindah ke node sebelumnya
            curr = curr.prev_1015;

        }
        //cetak spasi
        System.out.println();
    }
    public static void main(String[] args) {
        //cetak DLL
        NodeDLL_2511531015 head = new NodeDLL_2511531015(1);
        NodeDLL_2511531015 second = new NodeDLL_2511531015(2);
        NodeDLL_2511531015 third = new NodeDLL_2511531015(3);

        head.next_1015 = second;
        second.prev_1015 = head;
        second.next_1015 = third;
        third.prev_1015 = second;

        System.out.println("penelusuran maju:");
        forwardTraversal(head);

        System.out.println("penelusuran mundur:");
```

1. Package Declaration

package pekan6_2511531015;

Kode ini mendeklarasikan package tempat kelas ini berada, yaitu pekan6_2511531015.

2. Class PenulusuranDLL_2511531015

Kelas PenulusuranDLL_2511531015 berfungsi untuk melakukan penelusuran pada Doubly Linked List (DLL). Dalam DLL, setiap node memiliki dua pointer:

- **next_1015**: Menunjuk ke node berikutnya.
- **prev_1015**: Menunjuk ke node sebelumnya.

2.1 Fungsi forwardTraversal

Fungsi ini digunakan untuk melakukan penelusuran maju, yaitu dari **head** (node pertama) hingga **tail** (node terakhir).

```
static void forwardTraversal(NodeDLL_2511531015 head) {
    NodeDLL_2511531015 curr = head;
    while (curr != null) {
        System.out.print(curr.data_1015 + " <-> ");
```

```
        curr = curr.next_1015;  // Pindah ke node berikutnya
    }
    System.out.println();  // Menambahkan baris baru setelah traversal selesai
}
```

Langkah-langkah:

1. Mulai traversal dari node head.
2. Selama curr (current node) tidak null (berarti masih ada node berikutnya):
    - Cetak data dari node saat ini.
    - Pindah ke node berikutnya menggunakan pointer next_1015.
3. Setelah mencapai akhir list, cetak baris baru untuk memisahkan output.

2.2 Fungsi backwardTraversal

Fungsi ini digunakan untuk melakukan penelusuran mundur, yaitu dari **tail** (node terakhir) kembali ke **head** (node pertama).

```
static void backwardTraversal(NodeDLL_2511531015 tail) {
    NodeDLL_2511531015 curr = tail;
    while (curr != null) {
        System.out.print(curr.data_1015 + " <-> ");
        curr = curr.prev_1015;  // Pindah ke node sebelumnya
    }
    System.out.println();  // Menambahkan baris baru setelah traversal selesai
}
```

Langkah-langkah:

1. Mulai traversal dari node tail (node terakhir).
2. Selama curr (current node) tidak null (berarti masih ada node sebelumnya):
    - Cetak data dari node saat ini.
    - Pindah ke node sebelumnya menggunakan pointer prev_1015.
3. Setelah mencapai awal list, cetak baris baru untuk memisahkan output.

3. Method main

Pada method main, kita mendemonstrasikan cara kerja fungsi penelusuran maju dan mundur pada DLL.

```
public static void main(String[] args) {
    NodeDLL_2511531015 head = new NodeDLL_2511531015(1);
    NodeDLL_2511531015 second = new NodeDLL_2511531015(2);
    NodeDLL_2511531015 third = new NodeDLL_2511531015(3);

    head.next_1015 = second;
    second.prev_1015 = head;
    second.next_1015 = third;
    third.prev_1015 = second;

    System.out.println("penelusuran maju:");
    forwardTraversal(head);  // Menampilkan DLL dari head ke tail

    System.out.println("penelusuran mundur:");
    backwardTraversal(third);  // Menampilkan DLL dari tail ke head
}
```

Langkah-langkah:

1. Membuat tiga node dalam DLL: head (data = 1), second (data = 2), dan third (data = 3).
2. Menghubungkan node satu dengan lainnya menggunakan pointer next_1015 dan prev_1015.
    - head menunjuk ke second.
    - second menunjuk ke third.
    - second memiliki pointer kembali ke head, dan third memiliki pointer kembali ke second.

3. Mencetak **penelusuran maju** dengan memanggil fungsi forwardTraversal(head) yang menampilkan list dari head ke tail.
4. Mencetak **penelusuran mundur** dengan memanggil fungsi backwardTraversal(third) yang menampilkan list dari tail ke head.

```java
package pekan6_2511531015;

public class HapusDLL_2511531015 {
// fungsi menghapus node awal
    public static NodeDLL_2511531015 delHead(NodeDLL_2511531015 head) {
        if (head == null) {
            return null; }
            NodeDLL_2511531015 temp = head;
            head = head.next_1015 ;
            if(head != null) {
                head.prev_1015 = null; }
            return head;


            }
            //fungsi menghapus di akhir
    public static NodeDLL_2511531015 delLast(NodeDLL_2511531015 head) {
        if (head == null) {
            return null; }
            if (head.next_1015 == null) {
                return null; }
            NodeDLL_2511531015 curr = head;
            while (curr.next_1015 != null) {
                curr = curr.next_1015 ;

            }
            //update pointer previuos node
            if (curr.prev_1015 != null) {
                curr.prev_1015.next_1015 = null; }
            return head;
        }
    //fungsi menghapus node posisi tertentu
    public static NodeDLL_2511531015 delPos(NodeDLL_2511531015 head, int pos) {
        //jia dll kosong
        if (head == null) {
            return head; }
        NodeDLL_2511531015 curr = head;
        //telusuri sampai ke node yang akan dihapus
        for (int i = 1; curr != null && i < pos; i++) {
            curr = curr.next_1015 ; }
            //jika posisi tidak ditemukan
            if(curr == null) {
                return head; }
            //update pointer
            if (curr.prev_1015 != null) {
                curr.prev_1015.next_1015 = curr.next_1015 ;  }
            if (curr.next_1015 != null) {
                curr.next_1015.prev_1015 =curr.prev_1015; }
```

```java
            //jika yang dihapus head
            if (head == curr) {
                head = curr.next_1015; }
            return head;
            }
        //fungsi mencetak DLL
        public static void printList(NodeDLL_2511531015 head) {
            NodeDLL_2511531015 curr = head;
            while (curr != null) {
                System.out.print(curr.data_1015 + " ");
                curr = curr.next_1015;

            }
            System.out.println();

        }
        public static void main(String[] args) {
            //buat sebuah DLL
            NodeDLL_2511531015 head = new NodeDLL_2511531015(1);
            head.next_1015 = new NodeDLL_2511531015(2);
            head.next_1015.prev_1015 = head;
            head.next_1015.next_1015 = new NodeDLL_2511531015(3);
            head.next_1015.next_1015.prev_1015 = head.next_1015;
            head.next_1015.next_1015.next_1015 = new NodeDLL_2511531015(4);
            head.next_1015.next_1015.next_1015.prev_1015 = head.next_1015.next_1015;
            head.next_1015.next_1015.next_1015.next_1015 = new NodeDLL_2511531015(5);
            head.next_1015.next_1015.next_1015.next_1015.prev_1015 =    head.next_1015.next_1015.next_1015;

            System.out.print("DLL awal: ");
            printList(head);

            System.out.print("Setelah head dihapus: ");
            head = delHead(head);
            printList(head);

            System.out.print("Setelah node terakhir dihapus: ");
            head = delLast(head);
            printList(head);

            System.out.print("Menghapus node ke 2: ");
            head = delPos(head, 2);

            printList(head);
        }
        }
```

1. Package Declaration

package pekan6_2511531015;

Kode ini mendeklarasikan package tempat kelas ini berada, yaitu pekan6_2511531015.

2. Class HapusDLL_2511531015

Kelas HapusDLL_2511531015 berisi metode-metode untuk menghapus node pada **Doubly Linked List (DLL)**.

2.1 Fungsi delHead (Menghapus Node di Awal)

Fungsi ini digunakan untuk menghapus node pertama (head) dalam DLL.

```
public static NodeDLL_2511531015 delHead(NodeDLL_2511531015 head) {
    if (head == null) {
        return null;  // Jika DLL kosong, return null
    }
    NodeDLL_2511531015 temp = head;
    head = head.next_1015;  // Pindahkan head ke node berikutnya
    if (head != null) {
        head.prev_1015 = null;  // Set prev dari node baru menjadi null
    }
    return head;  // Kembalikan head yang baru
}
```

Langkah-langkah:

1. Jika DLL kosong (head == null), fungsi akan mengembalikan null.
2. Jika DLL tidak kosong, node pertama akan dihapus dan head akan dipindahkan ke node berikutnya.
3. Pointer prev_1015 dari node yang baru menjadi null karena sudah tidak ada node sebelumnya.
4. Fungsi mengembalikan head yang baru.

2.2 Fungsi delLast (Menghapus Node di Akhir)

Fungsi ini digunakan untuk menghapus node terakhir (tail) dalam DLL.

```
public static NodeDLL_2511531015 delLast(NodeDLL_2511531015 head) {
    if (head == null) {
        return null;  // Jika DLL kosong, return null
    }
    if (head.next_1015 == null) {
```

```
        return null;  // Jika hanya ada satu node, hapus node tersebut
    }
    NodeDLL_2511531015 curr = head;
    while (curr.next_1015 != null) {
        curr = curr.next_1015;  // Menemukan node terakhir
    }
    if (curr.prev_1015 != null) {
        curr.prev_1015.next_1015 = null;  // Set next node dari node sebelumnya menjadi null
    }
    return head;  // Kembalikan head yang tetap
}
```

Langkah-langkah:

1. Jika DLL kosong (head == null), fungsi mengembalikan null.
2. Jika hanya ada satu node dalam DLL, maka node tersebut akan dihapus.
3. Fungsi melakukan traversal untuk mencapai node terakhir.
4. Pointer next_1015 dari node sebelumnya diatur ke null, sehingga node terakhir dihapus.
5. Fungsi mengembalikan head yang tetap.

2.3 Fungsi delPos (Menghapus Node pada Posisi Tertentu)

Fungsi ini digunakan untuk menghapus node pada posisi tertentu dalam DLL.

```
public static NodeDLL_2511531015 delPos(NodeDLL_2511531015 head, int pos) {
    if (head == null) {
        return head;  // Jika DLL kosong, return head (null)
    }
    NodeDLL_2511531015 curr = head;
    for (int i = 1; curr != null && i < pos; i++) {
        curr = curr.next_1015;  // Menemukan node yang akan dihapus
    }
    if (curr == null) {
        return head;  // Jika posisi tidak ditemukan, return head
    }
    if (curr.prev_1015 != null) {
        curr.prev_1015.next_1015 = curr.next_1015;  // Menghubungkan node sebelumnya ke node
berikutnya
    }
    if (curr.next_1015 != null) {
        curr.next_1015.prev_1015 = curr.prev_1015;  // Menghubungkan node berikutnya ke node
```

```
sebelumnya
    }
    if (head == curr) {
        head = curr.next_1015;  // Jika node yang dihapus adalah head, perbarui head
    }
    return head;  // Kembalikan head yang tetap
}
```

Langkah-langkah:

1. Jika DLL kosong (head == null), fungsi mengembalikan head yang null.
2. Fungsi melakukan traversal untuk menemukan node pada posisi yang ditentukan.
3. Setelah menemukan node yang akan dihapus, pointer next_1015 dari node sebelumnya dan pointer prev_1015 dari node berikutnya diupdate.
4. Jika yang dihapus adalah head, maka head akan diperbarui ke node berikutnya.
5. Fungsi mengembalikan head yang telah diperbarui.

2.4 Fungsi printList (Mencetak DLL)

Fungsi ini digunakan untuk mencetak seluruh isi DLL.

```
public static void printList(NodeDLL_2511531015 head) {
    NodeDLL_2511531015 curr = head;
    while (curr != null) {
        System.out.print(curr.data_1015 + " ");
        curr = curr.next_1015;  // Menelusuri setiap node dalam DLL
    }
    System.out.println();  // Menambahkan baris baru setelah traversal selesai
}
```

Langkah-langkah:

1. Fungsi ini melakukan traversal pada DLL dari head hingga akhir list.
2. Mencetak nilai data_1015 dari setiap node yang dilewati.
3. Setelah traversal selesai, mencetak baris baru untuk memisahkan output.

3. Method main

Pada method main, kita mendemonstrasikan cara kerja fungsi-fungsi yang telah dibuat untuk menghapus node dalam DLL.

```
public static void main(String[] args) {
    NodeDLL_2511531015 head = new NodeDLL_2511531015(1);
    head.next_1015 = new NodeDLL_2511531015(2);
    head.next_1015.prev_1015 = head;
    head.next_1015.next_1015 = new NodeDLL_2511531015(3);
    head.next_1015.next_1015.prev_1015 = head.next_1015;
    head.next_1015.next_1015.next_1015 = new NodeDLL_2511531015(4);
    head.next_1015.next_1015.next_1015.prev_1015 = head.next_1015.next_1015;
    head.next_1015.next_1015.next_1015.next_1015 = new NodeDLL_2511531015(5);
    head.next_1015.next_1015.next_1015.next_1015.prev_1015 =
head.next_1015.next_1015.next_1015;

    System.out.print("DLL awal: ");
    printList(head);  // Mencetak DLL awal

    System.out.print("Setelah head dihapus: ");
    head = delHead(head);  // Menghapus node head
    printList(head);  // Mencetak DLL setelah node head dihapus

    System.out.print("Setelah node terakhir dihapus: ");
    head = delLast(head);  // Menghapus node terakhir
    printList(head);  // Mencetak DLL setelah node terakhir dihapus

    System.out.print("Menghapus node ke 2: ");
    head = delPos(head, 2);  // Menghapus node pada posisi ke-2
    printList(head);  // Mencetak DLL setelah node ke-2 dihapus
}
```

KESIMPULAN

Pada kode ini, berbagai operasi penghapusan pada **Doubly Linked List (DLL)** telah berhasil diimplementasikan, termasuk menghapus node di awal, di akhir, dan pada posisi tertentu dalam list. Fungsi delHead digunakan untuk menghapus node pertama (head) dari DLL, dan memperbarui head untuk menunjuk ke node berikutnya, dengan penanganan khusus jika list kosong. Fungsi delLast menghapus node terakhir (tail) dengan melakukan traversal hingga mencapai node terakhir dan mengatur pointer next_1015 dari node sebelumnya menjadi null.

Sementara itu, fungsi delPos memungkinkan penghapusan node pada posisi tertentu dengan cara menelusuri list hingga mencapai posisi yang diinginkan, kemudian memperbarui pointer next_1015 dan prev_1015 agar node sebelumnya dan sesudahnya saling terhubung kembali. Untuk memverifikasi hasil operasi, fungsi printList digunakan untuk mencetak isi DLL setelah setiap penghapusan. Dengan menggunakan pointer prev_1015 dan next_1015, DLL memberikan fleksibilitas dalam pengelolaan data, memungkinkan penambahan dan penghapusan node di berbagai posisi dengan efisien. Secara keseluruhan, kode ini menggambarkan bagaimana mengelola struktur data **Doubly Linked List** dengan operasi dasar yang esensial dalam pemrograman.